

Package ‘alloy’

August 11, 2026

Type Package

Title Unified Modeling Framework for Regression and Classification

Version 0.0.0.910

Description Consistent user interface to the most common regression and classification algorithms, such as random forest, neural networks, C5 trees and support vector machines, complemented with a handful of auxiliary functions, such as variable importance and a tuning function for the parameters.

Encoding UTF-8

Depends R (>= 4.6.0)

License GPL (>= 2)

LinkingTo Rcpp, RcppParallel, RcppArmadillo

Roxygen list(markdown = TRUE)

Config/testthat/edition 3

Imports stats,
MASS,
survival,
nnet,
pharos,
bedrock,
DescToolsX

Suggests e1071,
sandwich,
C50,
rpart,
randomForest,
naivebayes,
glmnet,
xgboost,
robustbase,
AER,
pscl,
rpart.plot,
relaimpo,
NeuralNetTools,
pROC,
boot,
VGAM,

MuMIn,
lme4,
haven,
foreign,
R.rsp,
testthat,
knitr,
rmarkdown

VignetteBuilder R.rsp, knitr

URL <https://andrisignorell.github.io/alloy/>,
<https://github.com/AndriSignorell/alloy/>

BugReports <https://github.com/AndriSignorell/alloy/issues>

Config/roxygen2/version 8.0.0

LazyData true

Contents

Admit	3
Apt	3
bestCut	4
bestTree	5
BioChemists	5
coefCI	6
coeffDiffCI	7
confint.roc	8
Contraception	8
cParam	9
Fish	10
fitMod	11
IceCream	12
Lahigh	13
leafRates	13
lift	14
model.matrix.gls	16
node	17
Ologit	17
Pima	18
plot.rpart	19
plot.varImp	20
predict.FitMod	21
print.FitMod	23
pseudoR2	25
refLevel	27
response	28
roc	29
rSq	30
rules	31
splits	32
splitTrainTest	33

<i>Admit</i>	3
tMod	34
tmodSummary	35
varImp	36
vif	37
Whas100	38
Index	39

Admit	<i>Graduate school admissions</i>
-------	-----------------------------------

Description

Admissions decisions for 400 applicants to a graduate programme, used to illustrate binary logistic regression.

Usage

`Admit`

Format

A data frame with 400 rows and 4 variables:

admit Binary outcome: 1 = admitted, 0 = not admitted.

gre GRE score.

gpa Grade point average.

rank Factor (1–4): prestige of undergraduate institution.

Source

<https://stats.oarc.ucla.edu/r/dae/logit-regression/>

Apt	<i>Academic aptitude scores (tobit example)</i>
-----	---

Description

Scores on an academic aptitude test for 200 high school students, right-censored at 800. Used to illustrate tobit regression.

Usage

`Apt`

Format

A data frame with 200 rows and 4 variables:

apt Academic aptitude score (censored at 800).

id ID for the student

read Reading score.

math Math score.

prog Factor: academic programme ("academic", "general", "vocational").

Source

<https://stats.oarc.ucla.edu/stata/dae/tobit-analysis/>

bestCut	<i>Best cut-point of an ROC curve</i>
---------	---------------------------------------

Description

A thin convenience wrapper around [coords](#) that returns the optimal cut-point of an ROC curve together with the corresponding sensitivity and specificity.

Usage

```
bestCut(x, method = c("youden", "closest.topleft"))
```

Arguments

<code>x</code>	An object of class "roc" as returned by roc .
<code>method</code>	Character string specifying the optimality criterion. "youden" (default) maximises the Youden index $J = \text{sensitivity} + \text{specificity} - 1$; "closest.topleft" minimises the Euclidean distance to the top-left corner (0, 1) of the ROC space.

Value

A named numeric vector with elements threshold, specificity, and sensitivity at the optimal cut-point. When ties exist, [coords](#) may return multiple columns; the result is then a matrix (one column per tied optimum).

See Also

[coords](#), [roc](#)

Other roc: [lift\(\)](#), [roc\(\)](#)

Examples

```
library(pROC)
data(aSAH)
r <- roc(aSAH$outcome, aSAH$s100b)

bestCut(r)
bestCut(r, method = "closest.topleft")
```

bestTree	<i>Best tree size using the 1-SE rule</i>
----------	---

Description

Selects the most parsimonious tree whose cross-validated error is within one standard error of the minimum, following Breiman et al. (1984).

Usage

```
bestTree(x)
```

Arguments

x A fitted rpart object.

Value

A list with components:

leaves Number of terminal nodes in the best tree.

cp Complexity parameter to pass to prune().

References

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.

See Also

Other tree: [cParam\(\)](#), [leafRates\(\)](#), [node\(\)](#), [plot.rpart\(\)](#), [rules\(\)](#), [splits\(\)](#)

Examples

```
r <- rpart::rpart(Species ~ ., data = iris)
bt <- bestTree(r)
r.pruned <- rpart::prune(r, cp = bt$cp)
```

BioChemists	<i>Biochemists' publication counts</i>
-------------	--

Description

Publication counts for 915 biochemistry PhD students, used to illustrate zero-inflated and count regression models. Originally from the **pscl** package.

Usage

```
BioChemists
```

Format

A data frame with 915 rows and 6 variables:

art Number of articles published in the last three years of the PhD (count response).

fem Factor: gender ("Men", "Women").

mar Factor: marital status ("Single", "Married").

kid5 Number of children under age 6.

phd Prestige of PhD programme.

ment Number of articles published by the mentor in the last three years.

Source

Long, J. S. (1997). *Regression Models for Categorical and Limited Dependent Variables*. Sage.

coefCI	<i>Bootstrap Confidence Intervals for Linear Model Coefficients</i>
--------	---

Description

Computes bootstrap confidence intervals for regression coefficients from a linear model using a fast parallel implementation.

Usage

```
coefCI(
  fit,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  R = 2000,
  seed = NULL,
  ...
)
```

Arguments

<code>fit</code>	An object of class "lm".
<code>conf.level</code>	Confidence level. Default is 0.95.
<code>sides</code>	Type of interval: "two.sided", "left", or "right".
<code>R</code>	Number of bootstrap samples.
<code>seed</code>	Optional random seed.
<code>...</code>	Further arguments (unused).

Details

Uses a nonparametric bootstrap (resampling observations). For each sample, the linear model is refitted and coefficients are stored.

Confidence intervals are based on empirical quantiles.

Value

A matrix with rows corresponding to coefficients and columns: est, lci, uci.

See Also

Other regression.utils: [pseudoR2\(\)](#), [rSq\(\)](#), [refLevel\(\)](#), [response\(\)](#), [varImp\(\)](#), [vif\(\)](#)

Examples

```
fit <- lm(mpg ~ wt + hp, data = mtcars)
coefCI(fit)
```

coeffDiffCI	<i>Confidence Interval for the Difference of Two Regression Coefficients</i>
-------------	--

Description

Computes the Wald confidence interval for the difference $b_2 - b_1$ of two coefficients of a fitted linear model, using $Var(b_2 - b_1) = Var(b_1) + Var(b_2) - 2Cov(b_1, b_2)$ and the t-distribution with the model's residual degrees of freedom.

Usage

```
coeffDiffCI(
  x,
  coeff,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  vcov. = vcov
)
```

Arguments

<code>x</code>	a fitted model object of class "lm".
<code>coeff</code>	two coefficients, given as names or as (integer) positions in <code>coef(x)</code> . The difference is taken as second minus first, <code>coeff[2] - coeff[1]</code> .
<code>conf.level</code>	confidence level of the interval, default 0.95.
<code>sides</code>	a character string specifying the side of the confidence interval, one of "two.sided" (default), "left", or "right".
<code>vcov.</code>	covariance matrix of the coefficient estimates, or a function to compute it from <code>x</code> (default vcov). Supply e.g. <code>sandwich::vcovHC</code> for heteroskedasticity-robust intervals.

Value

a named numeric vector with components `diff`, `lci`, and `uci`.

Examples

```
fit <- lm(mpg ~ cyl + disp + hp, data = mtcars)
coeffDiffCI(fit, c("cyl", "hp"))
```

confint.roc	Confidence intervals for ROC curve coordinates
-------------	--

Description

Computes confidence intervals for sensitivity, specificity, or other coordinates of an ROC curve via [ci.coords](#).

Usage

```
## S3 method for class 'roc'
confint(object, parm, level = 0.95, x = "best", ...)
```

Arguments

object	An object of class "roc" as returned by roc or roc .
parm	Currently unused; present for compatibility with the generic confint .
level	Confidence level. Default is 0.95.
x	Coordinate at which to evaluate the confidence interval. Either a numeric value (e.g. a threshold or specificity value) or one of the special strings accepted by ci.coords : "best" (default), "all", or "local maximas".
...	Further arguments passed to ci.coords .

Value

A "ci.coords" object as returned by [ci.coords](#).

See Also

[ci.coords](#), [roc](#)

Examples

```
fitLogit <- fitMod(admit ~ gre + gpa + rank, Admit, fitfn = "logit")
r <- roc(fitLogit)
confint(r)                # CI at best cut-point
confint(r, x = 0.5)       # CI at specificity = 0.5
```

Contraception	Contraceptive use in Bangladesh
---------------	---------------------------------

Description

Data on contraceptive use by women in urban and rural areas of Bangladesh, from the 1988 Bangladesh Fertility Survey. Commonly used to illustrate mixed-effects logistic regression with a district-level random effect.

Usage

Contraception

Format

A data frame with 1934 rows and 6 variables:

woman Factor: identifying code for each woman.

district Factor: identifying code for each district (used as grouping variable in mixed models).

use Factor: contraceptive use at time of survey ("N" = no, "Y" = yes).

livch Ordered factor: number of living children at time of survey. Levels are 0, 1, 2, 3+.

age Numeric: age of woman at time of survey (years), centred around the mean.

urban Factor: type of region of residence ("urban", "rural").

Source

<https://www.bristol.ac.uk/cmm/learning/mmsoftware/data-rev.html>

References

Huq, N. M., and Cleland, J. (1990). *Bangladesh Fertility Survey 1989 (Main Report)*. Dhaka: National Institute of Population Research and Training.

Examples

```
fitLogitMixed <- fitMod(use ~ age + urban + (1 | district),
                        data = Contraception, fitfn = "logitMixed")
fitLogitMixed
fitLogitMixed |> print(output = "or")
```

cParam

Complexity parameter table for an rpart tree

Description

A lightweight wrapper around `printcp` and `plotcp` that stores the CP table and the minimum cross-validated error CP as a structured object.

Usage

```
cParam(x, ...)
```

Arguments

`x` A fitted rpart object.

`...` Currently unused.

Value

An object of class "CP" with components `cp` (the full CP table), `mincp` (CP at minimum xerror), and `x` (the original rpart object).

See Also

[bestTree](#), [printcp](#), [plotcp](#)

Other tree: [bestTree\(\)](#), [leafRates\(\)](#), [node\(\)](#), [plot.rpart\(\)](#), [rules\(\)](#), [splits\(\)](#)

Examples

```
r <- rpart::rpart(Species ~ ., data = iris)
cp <- cParam(r)
cp
plot(cp)
```

Fish

A dataset containing information about 250 groups of visitors to a state park. This dataset is the standard textbook example for demonstrating Zero-Inflated Poisson (ZIP) and Zero-Inflated Negative Binomial (ZINB) regression models, as it contains an excess number of zeros in the count response variable.

Description

A dataset containing information about 250 groups of visitors to a state park. This dataset is the standard textbook example for demonstrating Zero-Inflated Poisson (ZIP) and Zero-Inflated Negative Binomial (ZINB) regression models, as it contains an excess number of zeros in the count response variable.

Usage

Fish

Format

A data frame with 250 rows and 6 variables:

nofish Indicator for whether the group did not fish (1 = did not fish, 0 = fished).

livebait Indicator for whether live bait was used (1 = yes, 0 = no).

camper Indicator for whether the group brought a camper to the park (1 = yes, 0 = no).

persons Total number of people in the group.

child Number of children in the group.

xb Linear predictor value for the count part of the model (generated simulation variable).

zg Linear predictor value for the zero-inflation part of the model (generated simulation variable).

count The number of fish caught (count response variable; target containing excess zeros).

Source

<https://stats.oarc.ucla.edu/r/dae/zip/>

fitMod	<i>Fit a statistical or machine-learning model with automatic method selection</i>
--------	--

Description

A unified interface for fitting a wide range of regression and classification models. When `fitfn` is omitted the appropriate method is chosen automatically from the type of the response variable. The return value is always an object of class "FitMod" layered on top of the original model object, so all standard methods (`predict`, `print`, `coef`, ...) continue to work.

Usage

```
fitMod(formula, data, ..., subset, na.action, fitfn = NULL)
```

Arguments

<code>formula</code>	A two-sided model formula.
<code>data</code>	A data frame containing the variables in <code>formula</code> .
<code>...</code>	Additional arguments passed to the underlying fitting function.
<code>subset</code>	An optional vector specifying a subset of observations. Only supported for fitting functions that accept a subset argument (and for "glmnet" and "xgboost", where it is applied when the design matrix is built).
<code>na.action</code>	A function for handling missing values, passed to the underlying fitting function (or to <code>model.frame</code> for "glmnet" and "xgboost"). If not supplied, the default of the respective fitting function applies (usually <code>na.omit</code>).
<code>fitfn</code>	Character string naming the fitting method. One of "lm", "logit", "poisson", "quasipoisson", "gamma", "negbin", "polr", "lmrob", "tobit", "zeroinfl", "multinom", "nnet", "rpart", "C5.0", "lda", "qda", "svm", "naiveBayes", "randomForest", "glmnet", "xgboost", "coxph", "weibull", "exponential", "lognormal", "loglogistic", "lmMixed", "logitMixed", "poissonMixed", "negbinMixed", "gammaMixed". If NULL (default) the method is chosen automatically.

Details

Automatic method selection uses the following heuristic: a dichotomous response (exactly two distinct values, factor/logical or numeric coded as 0/1) is fitted with "logit", an ordered factor with "polr", an unordered factor with "multinom", a non-negative integer response with "poisson", and any other numeric response with "lm". Note that integer storage does not necessarily mean count data – data import functions often return integer columns for metric variables. The chosen method is always reported via `message()`; supply `fitfn` explicitly to override the heuristic.

Value

An object of class `c("FitMod", <original class>)`. For `xgboost` and `lme4` models, a list of class `c("FitMod", "FitMod.xgboost")` or `c("FitMod", "FitMod.lme4")` wrapping the original model object in `$model`. For "glmnet" and "xgboost" the result additionally stores `terms`, `xlev` and `x_train`, so that `predict()` can rebuild design matrices for new data with the factor levels of the training data.

See Also

Other modelling: `predict.FitMod()`, `print.FitMod()`

Examples

```
# Auto-detection: numeric response -> lm
fitMod(Sepal.Length ~ ., data = iris)

# factor response -> multinom
if (requireNamespace("nnet", quietly = TRUE)) {
  fitMod(Species ~ ., data = iris)
}

# Explicit method
if (requireNamespace("rpart", quietly = TRUE)) {
  fitMod(Species ~ ., data = iris, fitfn = "rpart")
}

# Mixed models
if (requireNamespace("lme4", quietly = TRUE)) {
  fitMod(Reaction ~ Days + (1 | Subject), lme4::sleepstudy,
    fitfn = "lmMixed")
}
```

IceCream

*Ice cream flavour preference***Description**

Preference data for 200 respondents choosing between three ice cream flavours, used to illustrate multinomial logistic regression and machine learning classifiers.

Usage

```
IceCream
```

Format

A data frame with 200 rows and 4 variables:

id Respondent identifier.

ice_cream Factor: preferred flavour ("vanilla" is reference, "chocolate", "strawberry").

video Hours of video watched per week.

puzzle Puzzle score.

female Binary: 1 = female, 0 = male.

Source

<https://stats.oarc.ucla.edu/stata/output/multinomial-logistic-regression/>

Lahigh

*Los Angeles High School Attendance and Test Scores Data***Description**

A dataset containing attendance records, demographic information, and standardized test scores for 316 high school students from two schools in Los Angeles. This dataset is frequently used to demonstrate count data models like Poisson and Negative Binomial regression.

Usage

```
data("Lahigh")
```

Format

A data frame with 316 rows and 10 variables:

id Student identification number.

gender Student's gender (factor/categorical variable).

ethnic Student's ethnicity (categorical variable, e.g., Hispanic, White, Black, Asian).

school School attended by the student (Type 1 or Type 2).

mathpr Math percentile rank from the Comprehensive Tests of Basic Skills (CTBS).

langpr Language arts percentile rank from the Comprehensive Tests of Basic Skills (CTBS).

mathnce Math Normal Curve Equivalent (NCE) score.

langnce Language arts Normal Curve Equivalent (NCE) score.

biling Bilingual status or English proficiency level (e.g., LEP for Limited English Proficiency).

daysabs Number of days absent during the school year (count response).

Source

<https://stats.idre.ucla.edu/stat/data/lahigh.csv>

<https://stats.oarc.ucla.edu/stata/output/poisson-regression/>

leafRates

*Misclassification rates per leaf node***Description**

Computes the number of correctly and incorrectly classified observations in each terminal (leaf) node of a classification tree.

Usage

```
leafRates(x)
```

Arguments

x A fitted rpart classification object (i.e. `x$method != "anova"`).

Value

An object of class `c("leafRates", "list")` with components:

`node` node ids of the leaf nodes.

`freq` Integer matrix with columns "right" and "wrong".

`p.row` Row proportions of `freq`.

`mfreq` Total observations per leaf.

`mperc` Proportion of total observations per leaf.

See Also

Other tree: `bestTree()`, `cParam()`, `node()`, `plot.rpart()`, `rules()`, `splits()`

Examples

```
r <- rpart::rpart(Species ~ ., data = iris)
leafRates(r)
```

lift

Lift and gain table for a fitted model or predictor vector

Description

Computes lift and cumulative gain over the score distribution of a binary classifier. Lift quantifies how much better a model concentrates the positive class in the top-scored cases than random selection would, and is the standard evaluation view wherever only a limited fraction of the cases can be acted upon - direct marketing, fraud triage, churn prevention.

Usage

```
lift(x, resp = NULL, nBins = 10)
```

```
## S3 method for class 'Lift'
print(x, digits = 3, ...)
```

Arguments

<code>x</code>	Either a fitted binary classification model of class "FitMod" (in which case predicted probabilities and the response are extracted automatically), or a numeric vector of predicted probabilities / scores when <code>resp</code> is supplied.
<code>resp</code>	Optional factor or binary vector of true class labels. If NULL (default), <code>x</code> must be a "FitMod" object and the response is extracted via response .
<code>nBins</code>	Number of equally sized score groups. The default 10 yields the customary decile table.
<code>digits</code>	Number of significant digits used for printing.
<code>...</code>	Further arguments, currently unused.

Details

As in [roc](#), the second column of `predict(x, type = "prob")` is used as the predictor when `x` is a "FitMod" object - the positive class is therefore the second factor level of the response. For models with non-standard probability output, supply the predictor vector explicitly via `x` and `resp`.

Cases are ranked by decreasing score and cut into `nBins` groups of equal size. Within group i , lift is the hit rate divided by the overall base rate; cumulative lift uses the pooled hit rate over groups 1.. i . A cumulative lift of 2 at depth 0.2 means the top-scored fifth of the cases contains twice the share of positives that a random fifth would.

Ties in the score are broken by the sort order. With heavily tied scores

- a coarse tree with few distinct leaf probabilities, for instance - group boundaries fall inside tie groups and the per-group lift becomes correspondingly unstable; the cumulative curve is unaffected in expectation but jagged. Reduce `nBins` in that situation rather than interpreting individual bins.

Incomplete cases are dropped without an `na.rm` switch: a score/response pair is structurally unusable when either side is missing.

Lift and ROC rest on the same ranking; neither says anything about calibration. A model can show excellent lift while its predicted probabilities are systematically biased.

Value

An object of class "Lift", a data frame with one row per group and the columns:

<code>bin</code>	group number, 1 = highest scores.
<code>nObs</code>	number of observations in the group.
<code>nPos</code>	number of positives in the group.
<code>rate</code>	hit rate within the group.
<code>lift</code>	group hit rate divided by the base rate.
<code>depth</code>	cumulative share of all observations up to this group.
<code>cumPos</code>	cumulative number of positives.
<code>cumRate</code>	cumulative hit rate.
<code>cumLift</code>	cumulative hit rate divided by the base rate.
<code>gain</code>	cumulative share of all positives captured.

The attributes `baseRate`, `nObs`, `nPos` and `positive` carry the overall figures.

See Also

[roc](#), [bestCut](#), [response](#), `pharos::plotLift()`

Other roc: [bestCut\(\)](#), [roc\(\)](#)

Examples

```
fitLogit <- fitMod(admit ~ gre + gpa + rank, Admit, fitfn = "logit")
lift(fitLogit)

# Supply predictor and response directly
p <- predict(fitLogit)[, 2]
lift(p, resp = Admit$admit)
```

```
# Coarser grouping
lift(fitLogit, nBins = 5)
```

model.matrix.gls	<i>Model matrix for gls objects</i>
------------------	-------------------------------------

Description

Extracts the model matrix from a gls object by reconstructing it from the model formula and data.

Usage

```
## S3 method for class 'gls'
model.matrix(object, ...)
```

Arguments

object	A fitted gls model object (from nlme).
...	Additional arguments (currently ignored).

Details

This function provides an S3 method for `model.matrix()` for gls objects, which do not have a built-in method in base R.

The data is extracted from the original model call. This may fail if the data is not available in the evaluation environment.

Value

A model matrix corresponding to the fixed-effects design matrix.

Examples

```
## Not run:
library(nlme)
mod <- gls(distance ~ age, data = Orthodont)
model.matrix(mod)

## End(Not run)
```

node	<i>Structured node information for an rpart tree</i>
------	--

Description

Returns a structured list with detailed information about selected nodes of a fitted `rpart` tree, including split details, class counts, and probabilities.

Usage

```
node(x, node = NULL, type = c("all", "split", "leaf"), digits = 3L)
```

Arguments

<code>x</code>	A fitted <code>rpart</code> object (must have been fitted with <code>model = TRUE</code> and <code>y = TRUE</code>).
<code>node</code>	Character or numeric vector of node ids. If <code>NULL</code> (default) all nodes are returned.
<code>type</code>	Character string. One of "all" (default), "split" (internal nodes only), or "leaf" (terminal nodes only).
<code>digits</code>	Integer. Number of significant digits for split values. Default is 3.

Value

An object of class "node", a named list with one element per selected node. Each element contains: `id`, `vname`, `isleaf`, `nobs`, `group`, `ycount`, `yprob`, `nodeprob`, `complexity`, `tprint`, and (for split nodes) `sons`, `sons_n`, `primarysplits`, `surrogatesplits`.

See Also

[rules](#)

Other tree: [bestTree\(\)](#), [cParam\(\)](#), [leafRates\(\)](#), [plot.rpart\(\)](#), [rules\(\)](#), [splits\(\)](#)

Examples

```
r <- rpart::rpart(Species ~ ., data = iris)
node(r)
node(r, type = "leaf")
```

ologit	<i>Ordinal outcome: applying to graduate school</i>
--------	---

Description

Survey data on 400 students' likelihood of applying to graduate school, used to illustrate ordinal logistic regression.

Usage

```
ologit
```

Format

A data frame with 400 rows and 3 variables:

apply Likelihood of applying to graduate school (ordered factor response: 0 = "unlikely", 1 = "somewhat likely", 2 = "very likely").

pared Parental education indicator (1 = at least one parent has a graduate degree, 0 = otherwise).

public Type of undergraduate institution (1 = public school, 0 = private school).

gpa Student's cumulative undergraduate Grade Point Average (GPA).

Source

<https://stats.oarc.ucla.edu/r/faq/ologit-coefficients/>

<https://stats.idre.ucla.edu/stat/data/ologit.dta>

Pima	<i>Pima Indians diabetes data</i>
------	-----------------------------------

Description

Clinical measurements for 768 Pima Indian women, used to illustrate binary classification models. Missing values (originally coded as 0) have been replaced with NA for variables where 0 is physiologically implausible.

Usage

Pima

Format

A data frame with 768 rows and 9 variables:

pregnant Number of times pregnant.

glucose Plasma glucose concentration (2-hour oral glucose tolerance test).

pressure Diastolic blood pressure (mm Hg).

triceps Triceps skin fold thickness (mm).

insulin 2-hour serum insulin (μ U/ml).

mass Body mass index.

pedigree Diabetes pedigree function.

age Age (years).

diabetes Factor: "neg" (no diabetes) or "pos" (diabetes).

Source

<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

References

Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 261–265.

plot.rpart	<i>Plot an rpart tree using rpart.plot with node labels</i>
------------	---

Description

Overrides the default rpart plot and delegates to `rpart.plot`, optionally overlaying node id labels on each box.

Usage

```
## S3 method for class 'rpart'
plot(
  x = stop("no 'x' arg"),
  type = 2L,
  extra = "auto",
  under = FALSE,
  fallen.leaves = TRUE,
  digits = 2L,
  varlen = 0L,
  faclen = 0L,
  roundint = TRUE,
  cex = NULL,
  tweak = 1,
  clip.facs = FALSE,
  clip.right.labs = TRUE,
  snip = FALSE,
  box.palette = "auto",
  shadow.col = 0,
  node.labels = TRUE,
  ...
)
```

Arguments

<code>x</code>	A fitted rpart object.
<code>type</code>	Integer passed to <code>rpart.plot</code> . Default 2.
<code>extra</code>	Passed to <code>rpart.plot</code> . Default "auto".
<code>under</code>	Logical. Default FALSE.
<code>fallen.leaves</code>	Logical. Default TRUE.
<code>digits</code>	Integer. Default 2.
<code>varlen</code>	Integer. Default 0 (full names).
<code>faclen</code>	Integer. Default 0 (full factor labels).
<code>roundint</code>	Logical. Default TRUE.
<code>cex</code>	Numeric or NULL.
<code>tweak</code>	Numeric. Default 1.
<code>clip.facs</code>	Logical. Default FALSE.
<code>clip.right.labs</code>	Logical. Default TRUE.

snip	Logical. Default FALSE.
box.palette	Character or list. Default "auto" uses the package palette.
shadow.col	Colour of node shadows. Default 0 (none).
node.labels	Logical. If TRUE (default), node ids are printed above each box.
...	Further arguments passed to rpart.plot.

Value

Invisibly returns the list returned by rpart.plot.

See Also

Other tree: [bestTree\(\)](#), [cParam\(\)](#), [leafRates\(\)](#), [node\(\)](#), [rules\(\)](#), [splits\(\)](#)

plot.varImp	<i>Cleveland dot plot for variable importance</i>
-------------	---

Description

Displays a Cleveland dot plot of variable importance scores as returned by [varImp](#). Variables are ordered by importance (most important at the top).

Usage

```
## S3 method for class 'varImp'
plot(
  x,
  main = "Variable Importance",
  xlab = "Importance",
  pch = 16,
  col = "steelblue",
  cex = 1.2,
  ...
)
```

Arguments

x	An object of class "varImp" as returned by varImp .
main	Character string for the plot title. Default is "Variable Importance".
xlab	Character string for the x-axis label. Default is "Importance".
pch	Plotting character. Default is 16 (filled circle).
col	Colour of the points and reference lines. Default is "steelblue".
cex	Numeric: point size. Default is 1.2.
...	Further arguments passed to dotchart .

Value

Invisibly returns x.

See Also[varImp](#)**Examples**

```
fitRf <- fitMod(ice_cream ~ video + puzzle + female,
               IceCream, fitfn = "randomForest")
plot(varImp(fitRf))
```

predict.FitMod

*Predict method for FitMod objects***Description**

Unified predict interface for all models fitted via [fitMod](#). For regression and survival models the predicted values are returned as a numeric vector. For classification models either class probabilities, predicted classes, or both are returned as a data.frame with consistent column names across all model types.

Usage

```
## S3 method for class 'FitMod'
predict(
  object,
  newdata = NULL,
  output = c("prob", "class", "both", "where", "leaf"),
  s = "lambda.1se",
  type = NULL,
  ...
)
```

Arguments

object	A fitted model of class "FitMod".
newdata	Optional data frame of new observations. If omitted, fitted values on the training data are returned.
output	Character string controlling the output for classification models. One of "prob" (default), "class", or "both". For fitfn = "rpart" additionally "where" (row index of the predicted leaf in the tree frame) or "leaf" (node label of the predicted leaf) are available. Ignored for regression and survival models.
s	For fitfn = "glmnet" only: the value of the penalty parameter λ at which predictions are made. Passed to predict.cv.glmnet . Default is "lambda.1se".
type	For regression models (lm, glm, etc.): the type argument passed to the underlying predict method. If not supplied, "response" is used, so that predictions are returned on the response scale both with and without newdata. For Cox models the default is "risk"; for parametric survival models (incl. tobit) the default is "response". Ignored for classification models (use output instead).
...	Further arguments passed to the underlying predict method.

Details

Whether a model is treated as classification or regression is decided from the fitted object itself (e.g. `rpart$method`, `randomForest$type`, the family/objective stored by `fitMod()` for `glmnet`/`xgboost`), not from the fitting method alone.

For classification models the column order of probability outputs is always aligned with the factor levels of the response variable, regardless of which model type is used. This ensures that `predict(fitLogit)` and `predict(fitRf)` return columns in the same order.

Models that require explicit newdata even for training-data predictions (e.g. `svm`, `C5.0`, `randomForest`) are handled transparently via an internal helper. Note that for `randomForest` this returns in-sample (not out-of-bag) predictions, consistent with the fitted-values semantics of all other methods.

For `fitfn = "glmnet"` and `"xgboost"`, design matrices for newdata are rebuilt from the terms and factor levels of the training data, so new data may contain a subset of the training factor levels.

For `fitfn = "logit"`, calling `predict(object)` returns a two-column probability data.frame (like all other classifiers). To obtain the linear predictor (log-odds), use `predict(object, type = "link")`.

Value

Regression models A numeric vector of fitted/predicted values on the response scale. This includes regression variants of the machine-learning methods (`rpart` with `method = "anova"`, `randomForest`/`svm`/`nnet` regressions, `glmnet` with gaussian/poisson family, `xgboost` with a regression objective).

Survival models (`coxph`) A numeric vector of predicted risk scores (`type = "risk"` by default).

Parametric survival models (`weibull`, `exponential`, `lognormal`, `loglogistic`, `tobit`) A numeric vector of predicted survival times / expected responses (`type = "response"` by default).

Classification models `output = "prob"` A data.frame with one column per class containing predicted probabilities. Column names match the factor levels of the response variable.

`output = "class"` A data.frame with a single column class (factor) containing the predicted class.

`output = "both"` The probability columns and the class column combined in one data.frame.

See Also

`fitMod`, `print.FitMod`

Other modelling: `fitMod()`, `print.FitMod()`

Examples

```
# Regression
fitLm <- fitMod(Fertility ~ ., swiss)
head(predict(fitLm))

# Binary classification - probabilities
fitLogit <- fitMod(admit ~ gre + gpa + rank, Admit, fitfn = "logit")
head(predict(fitLogit))
head(predict(fitLogit, output = "both"))

# Multinomial classification
if (requireNamespace("nnet", quietly = TRUE)) {
  fitMult <- fitMod(ice_cream ~ video + puzzle + female,
    IceCream, fitfn = "multinom")
}
```

```

    head(predict(fitMult, output = "both"))
  }

  # Cox model - risk scores
  if (requireNamespace("survival", quietly = TRUE)) {
    fitCox <- fitMod(Surv(foltime, folstatus) ~ gender, Whas100,
                     fitfn = "coxph")
    head(predict(fitCox))

    # Parametric survival - expected survival time
    fitWei <- fitMod(Surv(foltime, folstatus) ~ gender + age, Whas100,
                     fitfn = "weibull")
    head(predict(fitWei))
  }

```

print.FitMod	<i>Print method for FitMod objects</i>
--------------	--

Description

Displays a formatted summary of a fitted model of class "FitMod", with confidence intervals, p-values, reference category headers for factor predictors, and model fit statistics. The output style is consistent across all supported model types and loosely follows Stata conventions.

Usage

```

## S3 method for class 'FitMod'
print(
  x,
  digits = 3,
  pdigits = 3,
  conf.level = 0.95,
  output = NULL,
  useProfile = FALSE,
  vcov = NULL,
  ...
)

```

Arguments

x	A fitted model of class "FitMod".
digits	Integer. Number of significant digits for estimates and confidence intervals. Default is 3.
pdigits	Integer. Number of digits for p-values. Default is 3.
conf.level	Numeric scalar in (0, 1). Confidence level for intervals. Default is 0.95.
output	Character string controlling the scale of the coefficient table. Supported values depend on the model type: "coef" Raw coefficients (default for lm, glm, lmrob, polr, parametric survival). "or" Odds ratios - exp(coef) - for logistic and ordinal models.

	"irr" Incidence rate ratios - $\exp(\text{coef})$ - for Poisson and negative binomial models.
	"hr" Hazard ratios (default for coxph).
	"lhr" Log hazard ratios for coxph.
	"tr" Time ratios - $\exp(\text{coef})$ - (default for parametric survival models).
	"genuine" Passes through to the original <code>summary()</code> / <code>print()</code> of the underlying model object.
	If NULL (default), an appropriate value is chosen automatically based on the model class.
useProfile	Logical. If TRUE and the model is a glm, profile-likelihood confidence intervals are computed via <code>confint</code> . Otherwise (default) Wald intervals via <code>confint.default</code> are used. Ignored for non-GLM models.
vcov	Character string specifying the type of heteroscedasticity-consistent covariance matrix to use for standard errors, e.g. "HC3" (recommended), "HC0", "HC1". Passed to <code>vcovHC</code> . Supported for lm, glm, and lmrob models; ignored with a message for all others. If NULL (default) the model's own standard errors are used.
...	Further arguments passed to the underlying print helper.

Details

Factor predictors are displayed with a header row showing the reference category and an overall p-value from `drop1`. Dummy-coded rows are indented below the header.

For negative binomial models an additional overdispersion block is printed showing the parameter $\alpha = 1/\theta$ (Stata convention) with a one-sided likelihood-ratio test against the Poisson model.

For quasi-Poisson and quasi-binomial models, pseudo- R^2 and AIC are not available and a note is displayed instead.

Value

Invisibly returns the result of `summary(x)`.

See Also

`fitMod`, `predict.FitMod`

Other modelling: `fitMod()`, `predict.FitMod()`

Examples

```
fitLm <- fitMod(Fertility ~ ., swiss)
print(fitLm)
print(fitLm, vcov = "HC3")

fitLogit <- fitMod(admit ~ gre + gpa + rank, Admit, fitfn = "logit")
print(fitLogit)
print(fitLogit, output = "or")
print(fitLogit, output = "or", vcov = "HC3")

fitPois <- fitMod(daysabs ~ mathnce + langnce + gender,
                  Lahigh, fitfn = "poisson")
print(fitPois, output = "irr")
```

```

fitCox <- fitMod(Surv(foltime, folstatus) ~ gender, Whas100,
                 fitfn = "coxph")
print(fitCox)
print(fitCox, output = "lhr")

fitWei <- fitMod(Surv(foltime, folstatus) ~ gender + age, Whas100,
                 fitfn = "weibull")
print(fitWei)
print(fitWei, output = "coef")
print(fitWei, output = "genuine")

```

pseudoR2

*Pseudo R-squared Measures for Regression Models***Description**

Computes a set of pseudo R-squared statistics for fitted regression models, including generalized linear models and related extensions.

Usage

```
pseudoR2(fit, which = "McFadden")
```

Arguments

<code>fit</code>	A fitted model object. Supported classes include: • <code>glm</code> • <code>nnet::multinom</code> • <code>MASS::polr</code> • <code>VGAM::vglm</code> (requires package VGAM)
<code>which</code>	Character vector specifying which statistic(s) to return. Defaults to "McFadden". Use "all" to return all measures.

Details

The following measures are available:

- **McFadden**: Likelihood ratio index.
- **McFaddenAdj**: Adjusted McFadden index (penalized for model complexity).
- **CoxSnell**: Maximum likelihood R-squared.
- **Nagelkerke**: Scaled Cox-Snell R-squared.
- **AldrichNelson**: Based on likelihood ratio.
- **VeallZimmermann**: Correction of Aldrich-Nelson.
- **McKelveyZavoina**: Latent variable R-squared (for logit/probit).
- **Efron**: Squared correlation between observed and predicted values.
- **Tjur**: Coefficient of discrimination (binomial models only).
- **AIC**: Akaike Information Criterion.

- **BIC**: Bayesian Information Criterion.
- **logLik**: Log-likelihood of fitted model.
- **logLik0**: Log-likelihood of null (intercept-only) model.
- **G2**: Likelihood ratio statistic.

Supported classes include:

- `glm`
- `nnet::multinom` (requires package **nnet**)
- `MASS::polr` (requires package **MASS**)
- `VGAM::vglm` (requires package **VGAM**)

Pseudo R-squared measures provide analogues to the coefficient of determination for models where ordinary R-squared is not defined, such as logistic or ordinal regression.

The null model is internally refitted as an intercept-only model using `model.frame(x)` to ensure consistency with the original data and handling of missing values.

For `vglm` objects, the package **VGAM** must be installed. The function uses `VGAM::predictvglm()` for obtaining linear predictors.

Some measures (e.g., McKelvey-Zavoina) depend on the link function and are only defined for logit and probit links.

Value

A named numeric vector with the requested pseudo R-squared measure(s).

Note

- Weighted models are supported only if weights are stored in the fitted model.
- Models fitted with aggregated responses (e.g., `cbind(success, failure)`) may yield incorrect results.
- For `vglm` models, ensure the model was fitted with `model = TRUE` to guarantee correct extraction of the model frame.

References

- McFadden, D. (1974). Conditional logit analysis of qualitative choice behavior.
- Cox, D. R., & Snell, E. J. (1989). Analysis of Binary Data.
- Nagelkerke, N. J. D. (1991). A note on a general definition of the coefficient of determination.
- Veall, M. R., & Zimmermann, K. F. (1996). Pseudo-R2 measures for some common limited dependent variable models.

See Also

Other regression utils: [coefCI\(\)](#), [rSq\(\)](#), [refLevel\(\)](#), [response\(\)](#), [varImp\(\)](#), [vif\(\)](#)

Examples

```
# Logistic regression example
data(mtcars)
mtcars$am <- factor(mtcars$am)

fit <- glm(am ~ wt + hp, data = mtcars, family = binomial)

pseudoR2(fit)
pseudoR2(fit, which = "Nagelkerke")
pseudoR2(fit, which = "all")
```

refLevel

Reference level of factor predictors in a model

Description

Extracts the reference (baseline) category for every factor predictor in a fitted model object. The reference level is derived from the contrast matrix that was actually used during fitting, so the result is correct even when base in `contr.treatment` is not 1 or when contrasts have been set globally via `options`.

Usage

```
refLevel(x)
```

Arguments

x A fitted model object with a `terms` attribute and a model data frame (e.g. objects of class `"lm"`, `"glm"`, `"lmerMod"`, or `"glmerMod"`).

Details

The function inspects `attr(model.matrix(x), "contrasts")` for each factor predictor.

- If the contrast is stored as a *character string* (e.g. `"contr.treatment"`), the first level of the factor is returned as the reference.
- If the contrast is stored as a *matrix*, the reference level is identified as the unique row whose entries are all zero (i.e. the row that does not map to any dummy column).

Contrasts other than treatment contrasts (e.g. `contr.sum`, `contr.helmert`) are not supported and trigger an informative error.

Value

A named character vector whose names are the factor predictor variables and whose values are the corresponding reference levels. Returns a zero-length named character vector when the model contains no factor predictors.

See Also

`contrasts`, `contr.treatment`

Other regression utils: `coefCI()`, `pseudoR2()`, `rSq()`, `response()`, `varImp()`, `vif()`

Examples

```
m <- lm(Sepal.Length ~ Species, data = iris)
refLevel(m)
# Species
# "setosa"

# Custom base level
iris2 <- iris
contrasts(iris2$Species) <- contr.treatment(3, base = 2)
m2 <- lm(Sepal.Length ~ Species, data = iris2)
refLevel(m2)
# Species
# "versicolor"
```

response	<i>Extract the response variable from a fitted model</i>
----------	--

Description

Retrieves the response vector from a fitted model object, with special handling for model classes that do not store a standard terms component (C5.0, rpart, naive_bayes). The name of the response variable is attached as the "response" attribute of the return value.

Usage

```
response(x, ...)
```

Arguments

x	A fitted model object. Supported classes include "lm", "glm", "multinom", "polr", "rpart", "C5.0", and "naive_bayes". Any other class is attempted via the default model.frame path.
...	Currently unused; reserved for future methods.

Value

The response vector as returned by [model.response](#), or a factor reconstructed from internal model slots for rpart. The attribute "response" carries the name of the response variable as a character string, or NA_character_ if it cannot be determined.

See Also

[model.response](#), [model.frame](#)

Other regression.utils: [coefCI\(\)](#), [pseudoR2\(\)](#), [rSq\(\)](#), [refLevel\(\)](#), [varImp\(\)](#), [vif\(\)](#)

roc

*ROC curve for a fitted model or predictor vector***Description**

Convenience wrapper around [roc](#) that accepts either a fitted "FitMod" binary classification model or a numeric predictor vector directly.

Usage

```
roc(x, resp = NULL, ...)
```

Arguments

x	Either a fitted binary classification model of class "FitMod" (in which case predicted probabilities and the response are extracted automatically), or a numeric vector of predicted probabilities / scores when resp is supplied.
resp	Optional factor or binary vector of true class labels. If NULL (default), x must be a "FitMod" object and the response is extracted via response .
...	Further arguments passed to roc .

Details

When x is a "FitMod" object, the second column of `predict(x, type = "prob")` is used as the predictor (i.e. the probability of the second factor level). For models with non-standard probability output, supply the predictor vector explicitly via x and resp.

Value

An object of class "roc" as returned by [roc](#).

See Also

[roc](#), [bestCut](#), [response](#)

Other roc: [bestCut\(\)](#), [lift\(\)](#)

Examples

```
fitLogit <- fitMod(admit ~ gre + gpa + rank, Admit, fitfn = "logit")
r <- roc(fitLogit)
plot(r)

# Supply predictor and response directly
p <- predict(fitLogit)[, 2]
r2 <- roc(p, resp = Admit$admit)
```

rSq

*R-squared of a Linear Model***Description**

Computes the (adjusted) R-squared of a linear model fitted via [lm](#), optionally together with a bootstrap confidence interval based on a parallel **RcppParallel** implementation.

Usage

```
rSq(
  fit,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  adjusted = TRUE,
  R = 2000,
  seed = NULL,
  ...
)
```

Arguments

<code>fit</code>	an object of class "lm".
<code>conf.level</code>	confidence level of the interval. Set to NA (default) to return the point estimate only, without bootstrapping.
<code>sides</code>	a character string naming the side on which the finite bound lies, must be one of "two.sided" (default), "left" or "right".
<code>adjusted</code>	logical; if TRUE (default) the adjusted R-squared is reported, otherwise the ordinary one.
<code>R</code>	number of bootstrap replicates, defaults to 2000.
<code>seed</code>	integer seed for the bootstrap. If NULL (default) the seed is drawn from R's random number stream, so that <code>set.seed()</code> governs the result.
<code>...</code>	further arguments, currently unused.

Details

If `conf.level` is NA (the default), the coefficient is taken directly from the fitted model and a single number is returned. Only when a confidence level is supplied is the bootstrap started.

The bootstrap resamples observations (pairs bootstrap): for each replicate the model is refitted on the resampled rows of the model matrix and the R-squared is recomputed. The interval is formed from the empirical quantiles of the resulting distribution. Degenerate bootstrap samples (singular design matrices) are discarded.

The point estimate follows the definition used by [summary.lm](#), which for models without intercept is based on the uncentred sum of squares and therefore not comparable to the value of a model with intercept. For adjusted R-squared

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n - i}{n - p}$$

is used, where p is the rank of the model and i is 1 for a model with intercept and 0 otherwise.

sides names the side on which the finite bound lies, so "left" yields $[lci, \infty)$ and "right" yields $(-\infty, uci]$, with the full α on that single side. This reverses the meaning the argument has in **DescTools**, where it names the direction of the alternative hypothesis. Since R-squared cannot exceed 1, the open side of a "left" interval is reported as 1 rather than as Inf; for "right" the open side is 0, or -Inf for the adjusted coefficient, which is not bounded below.

The bootstrap branch supports neither weighted models nor models without intercept, as the C++ routine centres unconditionally and ignores weights. Both cases are rejected with an error; the point estimate remains available via `conf.level = NA`.

Value

If `conf.level` is NA, a single numeric value. Otherwise a named numeric vector with the elements

est the (adjusted) R-squared of the fitted model.

lci the lower confidence limit.

uci the upper confidence limit.

See Also

[lm](#), [summary.lm](#)

Other regression.utils: [coefCI\(\)](#), [pseudoR2\(\)](#), [refLevel\(\)](#), [response\(\)](#), [varImp\(\)](#), [vif\(\)](#)

Examples

```
fit <- lm(mpg ~ wt + hp, data = mtcars)

rSq(fit)
## [1] 0.8148

rSq(fit, adjusted = FALSE)
## [1] 0.8268

rSq(fit, conf.level = 0.95, seed = 123)

rSq(fit, conf.level = 0.95, sides = "left", R = 1000, seed = 123)
```

rules

Decision rules for an rpart tree

Description

Extracts the decision path rules for selected nodes (or all nodes / leaves only) of a fitted rpart tree.

Usage

```
rules(x, node = NULL, leafonly = FALSE)
```

Arguments

<code>x</code>	A fitted <code>rpart</code> object.
<code>node</code>	Character or numeric vector of node ids to extract. If <code>NULL</code> (default) all nodes are returned.
<code>leafonly</code>	Logical. If <code>TRUE</code> only terminal (leaf) nodes are returned. Default is <code>FALSE</code> .

Value

An object of class "rules", a list with components `frame`, `ylevels`, `ds.size`, and `path`. Returns `NA` if no nodes match the selection.

See Also

[node](#), [path.rpart](#)

Other tree: [bestTree\(\)](#), [cParam\(\)](#), [leafRates\(\)](#), [node\(\)](#), [plot.rpart\(\)](#), [splits\(\)](#)

Examples

```
r <- rpart::rpart(Species ~ ., data = iris)
rules(r)
rules(r, leafonly = TRUE)
```

splits

Split labels for each node of an rpart tree

Description

Returns a two-column character matrix with the left and right split labels for every node of a fitted `rpart` tree. Leaf nodes are represented by empty strings.

Usage

```
splits(x)
```

Arguments

<code>x</code>	A fitted <code>rpart</code> object.
----------------	-------------------------------------

Value

A character matrix with columns "cutleft" and "cutright" and one row per node.

See Also

Other tree: [bestTree\(\)](#), [cParam\(\)](#), [leafRates\(\)](#), [node\(\)](#), [plot.rpart\(\)](#), [rules\(\)](#)

Examples

```
r <- rpart::rpart(Species ~ ., data = iris)
splits(r)
```

splitTrainTest

Split Data into Training and Test Sets

Description

Splits a vector, matrix or data frame into training and test subsets.

Usage

```
splitTrainTest(x, p = 0.1, logical = FALSE)
```

Arguments

x	An object to split. Typically a vector, matrix or data frame.
p	Proportion of observations assigned to the test set. Must be between 0 and 1. Default is 0.1.
logical	Logical; if TRUE, a logical index vector is returned instead of the split data objects. TRUE indicates observations belonging to the test set.

Details

Splitting data into training and test subsets is a common strategy for evaluating predictive models. The training set is used to fit the model, while the test set is used to assess predictive performance on unseen data. The function performs random sampling using R's global random number generator. To obtain reproducible results, users should call `set.seed()` prior to invoking this function. The function itself does not modify the random number generator state.

Value

If `logical = FALSE`, a list with elements:

train Training subset.

test Test subset.

If `logical = TRUE`, a logical vector of length `n`.

Examples

```
splitTrainTest(iris)

set.seed = 123
d <- splitTrainTest(iris, p = 0.2)

str(d$train)
str(d$test)

idx <- splitTrainTest(iris, logical = TRUE)
table(idx)
```

tMod

*Compare multiple statistical models***Description**

Creates a unified comparison object for multiple fitted models by extracting coefficients, confidence intervals, and model statistics.

Usage

```
tMod(..., FUN = NULL, order = NA, verbose = FALSE)

## S3 method for class 'TMod'
print(x, digits = 3, naForm = "-", verbose = NULL, ...)

## S3 method for class 'TMod'
plot(x, terms = NULL, intercept = FALSE, ...)
```

Arguments

...	Fitted model objects
FUN	Formatting function applied to coefficients
order	Optional ordering of models
verbose	Logical; if TRUE, show extended statistics
x	A "TMod" object
digits	Number of digits for printing
naForm	String used for missing values
terms	Optional character vector specifying which model terms should be displayed in the plot. If NULL, all terms are shown.
intercept	Logical; if FALSE (default), intercept terms are excluded from the plot.

Details

The function standardizes model output across different model classes (e.g. `lm`, `glm`, `coxph`, `gam`, `lmer`) using S3 methods implemented via `tmodSummary()`. This enables direct comparison of model coefficients and fit statistics in tabular and graphical form.

Model names are automatically derived from the call. If unavailable, default names are assigned.

Value

An object of class "TMod" with components:

- `m`: formatted coefficient table
- `mm`: model statistics
- `mall`: array of estimates and confidence intervals
- `terms`: mapping of model terms

See Also[tmodSummary](#)Other model.comparison: [tmodSummary\(\)](#)**Examples**

```
# --- Linear models ---
m1 <- lm(mpg ~ wt, data = mtcars)
m2 <- lm(mpg ~ wt + hp, data = mtcars)
tMod(m1, m2)

# --- Generalized linear models ---
g1 <- glm(am ~ wt, data = mtcars, family = binomial)
g2 <- glm(am ~ wt + hp, data = mtcars, family = binomial)
tMod(g1, g2)

# --- Survival models ---
if(requireNamespace("survival", quietly = TRUE)){
  library(survival)
  s1 <- coxph(Surv(time, status) ~ age, data = lung)
  s2 <- coxph(Surv(time, status) ~ age + sex, data = lung)
  tMod(s1, s2)
}
```

tmodSummary

*Extract model summaries for model comparison***Description**

S3 generic used internally by [tMod](#) to extract coefficients and model statistics from fitted models in a standardized format.

Usage

```
tmodSummary(x, ...)

## S3 method for class 'lm'
tmodSummary(x, conf.level = 0.95, ...)

## S3 method for class 'lmrob'
tmodSummary(x, conf.level = 0.95, ...)

## S3 method for class 'glm'
tmodSummary(x, conf.level = 0.95, useProfile = TRUE, ...)

## S3 method for class 'coxph'
tmodSummary(x, conf.level = 0.95, ...)

## S3 method for class 'gam'
tmodSummary(x, conf.level = 0.95, ...)

## S3 method for class 'lmer'
tmodSummary(x, conf.level = 0.95, ...)
```

Arguments

<code>x</code>	A fitted model object
<code>...</code>	Additional arguments passed to methods
<code>conf.level</code>	Confidence level for intervals
<code>useProfile</code>	Logical; use profile likelihood for CI (glm only)

Details

Each method returns a list with components `coef` (a data frame of coefficients and confidence intervals) and `statsx` (a named numeric vector of model statistics). These are combined by `tMod()` to enable comparison across different model types.

Value

A list with components:

- `coef`: data frame with columns `name`, `est`, `se`, `stat`, `p`, `lci`, `uci`
- `statsx`: named numeric vector of model statistics

See Also

[tMod](#)

Other model.comparison: [tMod\(\)](#)

<code>varImp</code>	<i>Variable importance for machine learning models</i>
---------------------	--

Description

Extracts and normalises variable importance scores from fitted machine learning models. Returns a `data.frame` sorted by importance, scaled to a 0–100 range.

Usage

```
varImp(x, scale = c("max", "sum", "none"), sort = TRUE, ...)
```

Arguments

<code>x</code>	A fitted model of class <code>"FitMod"</code> . Supported fitting functions: <code>"rpart"</code> , <code>"C5.0"</code> , <code>"randomForest"</code> , <code>"nnet"</code> , <code>"glmnet"</code> , <code>"xgboost"</code> .
<code>scale</code>	Character string controlling the scaling of importance scores. One of <code>"max"</code> (default: best predictor = 100, others relative), <code>"sum"</code> (scores sum to 100, interpretable as percentage share), or <code>"none"</code> (raw scores as returned by the underlying method).
<code>sort</code>	Logical. If <code>TRUE</code> (default), rows are sorted in descending order of importance.
<code>...</code>	Further arguments passed to the underlying importance method.

Value

A data.frame of class c("varImp", "data.frame") with columns:

variable Character: predictor name.

importance Numeric: importance score (0–100 if scale = TRUE).

See Also

[plot.varImp](#)

Other regression.utils: [coefCI\(\)](#), [pseudoR2\(\)](#), [rSq\(\)](#), [refLevel\(\)](#), [response\(\)](#), [vif\(\)](#)

Examples

```
fitRf <- fitMod(ice_cream ~ video + puzzle + female,
               IceCream, fitfn = "randomForest")
vi <- varImp(fitRf)
vi
plot(vi)
```

vif

Variance Inflation Factors (VIF / GVIF)

Description

Computes variance inflation factors (VIF) or generalized VIFs (GVIF) for a fitted model. For multi-parameter terms (e.g., factors), GVIFs are returned along with a scaled version $GVIF^{1/(2 \cdot Df)}$.

Usage

```
vif(fit)
```

Arguments

fit A fitted model object. Currently supports objects of class `lm`, `glm`, and `glms`.

Details

The function is based on the implementation in the **car** package (Fox and Weisberg). GVIFs are computed from the correlation matrix of the model coefficients.

Interpretation:

- Values close to 1 indicate low multicollinearity
- Values > 5 or 10 may indicate problematic collinearity

Note that VIFs are only meaningful if the model includes an intercept. For models without intercept, a warning is issued.

Value

If all terms have 1 degree of freedom, a named numeric vector of VIFs. Otherwise, a matrix with columns:

GVIF Generalized variance inflation factor
Df Degrees of freedom for the term
GVIF^{1/(2*Df)} Scaled GVIF for comparability

See Also

Other regression utils: [coefCI\(\)](#), [pseudoR2\(\)](#), [rSq\(\)](#), [refLevel\(\)](#), [response\(\)](#), [varImp\(\)](#)

Examples

```
mod <- lm(mpg ~ wt + cyl, data = mtcars)
vif(mod)
```

Whas100

Worcester Heart Attack Study (WHAS100)

Description

Follow-up data for 100 patients hospitalised with acute myocardial infarction, used to illustrate Cox proportional hazards and parametric survival models.

Usage

```
Whas100
```

Format

A data frame with 100 rows and 14 variables:

id Subject identification number.
addate Admission date to the hospital.
foldate Follow-up date.
hosstay Length of hospital stay (in days).
foltime Follow-up time from hospital admission to follow-up date (in days).
folstatus Vital status at follow-up (0 = alive, 1 = dead; event indicator).
age Age at hospital admission (in years).
gender Gender of the patient (0 = male, 1 = female).
bmi Body Mass Index (kg/m²).
agex Age group (factor), derived from age via [cutAge](#).

Source

<https://stats.oarc.ucla.edu/stata/examples/asa2/>

References

Hosmer, D. W., Lemeshow, S., & May, S. (2008). *Applied Survival Analysis*, 2nd ed. Wiley.

Index

- * **bootstrap**
 - coefCI, 6
 - rSq, 30
- * **classification**
 - bestCut, 4
 - bestTree, 5
 - cParam, 9
 - fitMod, 11
 - leafRates, 13
 - lift, 14
 - node, 17
 - plot.rpart, 19
 - roc, 29
 - rules, 31
 - splits, 32
- * **confidence-interval**
 - coefCI, 6
 - rSq, 30
- * **data.split**
 - splitTrainTest, 33
- * **datasets**
 - Admit, 3
 - Apt, 3
 - BioChemists, 5
 - Contraception, 8
 - Fish, 10
 - IceCream, 12
 - Lahigh, 13
 - Ologit, 17
 - Pima, 18
 - Whas100, 38
- * **feature-selection**
 - varImp, 36
- * **model-evaluation**
 - bestCut, 4
 - lift, 14
 - pseudoR2, 25
 - roc, 29
 - tMod, 34
 - tmodSummary, 35
- * **model.comparison**
 - tMod, 34
 - tmodSummary, 35
- * **modelling**
 - bestTree, 5
 - cParam, 9
 - fitMod, 11
 - leafRates, 13
 - node, 17
 - plot.rpart, 19
 - predict.FitMod, 21
 - print.FitMod, 23
 - rules, 31
 - splits, 32
 - splitTrainTest, 33
 - varImp, 36
- * **multicollinearity**
 - vif, 37
- * **prediction**
 - lift, 14
 - predict.FitMod, 21
- * **regression.utils**
 - coefCI, 6
 - pseudoR2, 25
 - refLevel, 27
 - response, 28
 - rSq, 30
 - varImp, 36
 - vif, 37
- * **regression**
 - coefCI, 6
 - fitMod, 11
 - pseudoR2, 25
 - refLevel, 27
 - response, 28
 - rSq, 30
 - tMod, 34
 - tmodSummary, 35
 - vif, 37
- * **roc**
 - bestCut, 4
 - lift, 14
 - roc, 29
- * **tree**
 - bestTree, 5
 - cParam, 9

- leafRates, 13
- node, 17
- plot.rpart, 19
- rules, 31
- splits, 32
- Admit, 3
- Apt, 3
- bestCut, 4, 15, 29
- bestCut(), 15, 29
- bestTree, 5, 10
- bestTree(), 10, 14, 17, 20, 32
- BioChemists, 5
- ci.coords, 8
- coefCI, 6
- coefCI(), 26–28, 31, 37, 38
- coeffDiffCI, 7
- confint, 8, 24
- confint.default, 24
- confint.roc, 8
- contr.treatment, 27
- Contraception, 8
- contrasts, 27
- coords, 4
- cParam, 9
- cParam(), 5, 14, 17, 20, 32
- cutAge, 38
- dotchart, 20
- drop1, 24
- Fish, 10
- fitMod, 11, 21, 22, 24
- fitMod(), 22, 24
- IceCream, 12
- Lahigh, 13
- leafRates, 13
- leafRates(), 5, 10, 17, 20, 32
- lift, 14
- lift(), 4, 29
- lm, 30, 31
- model.frame, 11, 28
- model.matrix.gls, 16
- model.response, 28
- na.omit, 11
- node, 17, 32
- node(), 5, 10, 14, 20, 32
- Ologit, 17
- options, 27
- path.rpart, 32
- Pima, 18
- plot.rpart, 19
- plot.rpart(), 5, 10, 14, 17, 32
- plot.TMod (tMod), 34
- plot.varImp, 20, 37
- plotcp, 9, 10
- predict.cv.glmnet, 21
- predict.FitMod, 21, 24
- predict.FitMod(), 12, 24
- print.FitMod, 22, 23
- print.FitMod(), 12, 22
- print.Lift (lift), 14
- print.TMod (tMod), 34
- printcp, 9, 10
- pseudoR2, 25
- pseudoR2(), 7, 27, 28, 31, 37, 38
- refLevel, 27
- refLevel(), 7, 26, 28, 31, 37, 38
- response, 14, 15, 28, 29
- response(), 7, 26, 27, 31, 37, 38
- roc, 4, 8, 15, 29, 29
- roc(), 4, 15
- rpart.plot, 19
- rSq, 30
- rSq(), 7, 26–28, 37, 38
- rules, 17, 31
- rules(), 5, 10, 14, 17, 20, 32
- splits, 32
- splits(), 5, 10, 14, 17, 20, 32
- splitTrainTest, 33
- summary.lm, 30, 31
- tMod, 34, 35, 36
- tMod(), 36
- tmodSummary, 35, 35
- tmodSummary(), 35
- varImp, 20, 21, 36
- varImp(), 7, 26–28, 31, 38
- vcov, 7
- vcovHC, 24
- vif, 37
- vif(), 7, 26–28, 31, 37
- Whas100, 38